Assignment 4: Heap Data Structures: Implementation, Analysis, and Applications

Fathiya Adan

University of Cumberlands

Algorithms and Data Structures (MSCS-532-B01)

Michael Solomon

March 22, 2026

1. Introduction

Heap data structures are fundamental tools in computer science because they support efficient priority-based operations and form the basis of important algorithms such as Heapsort and Priority Queues. A heap is a specialized binary tree that satisfies the heap property. In a max-heap, the value of each parent node is greater than or equal to the values of its children, while in a min-heap, the parent node is smaller than or equal to its children. Heaps are commonly implemented using arrays because this representation is compact and allows parent-child relationships to be computed directly using indices.

The objective of this assignment was to implement and analyze heap-based algorithms in Python. The first part focused on implementing the Heapsort algorithm and analyzing its behavior in terms of time and space complexity. This also included an empirical comparison of Heapsort with Quicksort and Merge Sort. The second part focused on implementing a Priority Queue using a binary heap and demonstrating how heap structures can be used in real-world scheduling applications.

This report presents the implementation details, complexity analysis, empirical observations, and application discussion for both parts of the assignment.

2. Heapsort Implementation

2.1 Overview of Heapsort

Heapsort is a comparison-based sorting algorithm that uses a binary heap data structure. The algorithm operates in two major phases:

1. Build a max-heap from the input array.
2. Repeatedly extract the maximum element from the heap and place it at the end of the array.

Because the largest element in a max-heap is always located at the root, the algorithm can efficiently move elements into sorted position one by one.

2.2 Algorithm Process

Heapsort operates in two main phases:

Phase 1: Build Max-Heap

The input array is converted into a max-heap. This is done by starting from the last non-leaf node and applying the heap property upward toward the root.

Phase 2: Extract Maximum

Once the heap is built:

- The root element (largest value) is swapped with the last element
- The heap size is reduced
- The heap property is restored

This process repeats until all elements are sorted.

3. Analysis of Heapsort

3.1 Time Complexity

Heapsort has a well-defined and consistent time complexity due to its structured operations.

Building the Heap

Although heapifying a single node may take $O(\log n)$, building the entire heap takes:

$$O(n)$$

This is because most nodes are located near the bottom of the tree and require minimal adjustments.

Extracting Elements

Each extraction requires restoring the heap, which takes $O(\log n)$. Since this is done for all elements:

$$O(n\log n)$$

Overall Complexity

$$O(n) + O(n\log n) = O(n\log n)$$

Thus:

- Best Case: $O(n\log n)$
- Average Case: $O(n\log n)$
- Worst Case: $O(n\log n)$

This makes Heapsort predictable and reliable compared to some other algorithms.

3.2 Space Complexity

Heapsort is an in-place algorithm, meaning it does not require additional memory proportional to the input size.

$$O(1)$$

Only a small number of variables are used, making it memory efficient.

3.3 Key Properties

- In-place algorithm
- Not stable
- Guaranteed performance $O(n\log n)$
- Suitable for memory-constrained environments

4. Comparison with Quicksort and Merge Sort

4.1 Experimental Setup

To evaluate performance, Heapsort was compared with Quicksort and Merge Sort using randomly generated arrays of different sizes:

- 1000 elements
- 5000 elements
- 10000 elements

Execution time was measured using Python's timing functions.

4.2 Results

Array Size	Heapsort (s)	Quicksort (s)	Merge Sort (s)
1000	0.00301	0.00230	0.00224
5000	0.01765	0.01259	0.01335
10000	0.03917	0.02676	0.02848

4.4 Discussion of Results

The results show that all three algorithms performed efficiently and that execution time increased as input size increased. This is consistent with the theoretical expectation of $O(n\log n)$growth.

However, the algorithms did not perform identically.

Quicksort

Quicksort was the fastest algorithm across all tested input sizes. This is often observed in practice because Quicksort has very good cache behavior and relatively low constant factors. Although its worst-case time complexity is $O(n^2)$, its average-case performance is usually excellent.

Merge Sort

Merge Sort performed slightly slower than Quicksort but faster than Heapsort. Its performance remained stable and predictable. Merge Sort is particularly strong because it always runs in $O(n\log n)$, but it requires additional memory for merging, which can increase overhead.

Heapsort

Heapsort was the slowest of the three algorithms in these experiments. Even though it also has $O(n\log n)$time complexity, it often performs more swaps and less cache-friendly memory access than Quicksort. These practical implementation costs make it slower in many real-world settings.

4.5 Interpretation

The empirical results confirm an important idea in algorithm analysis: theoretical complexity does not tell the entire story. All three algorithms are $O(n\log n)$, but implementation details, memory behavior, recursion overhead, and data access patterns affect actual execution time.

The observed ordering was:

$$\text{Quicksort} < \text{Merge Sort} < \text{Heapsort}$$

for all tested random datasets.

5. Priority Queue Implementation and Applications

5.1 Design Choice

For the second part of the assignment, a Priority Queue was implemented using a binary max-heap stored in a Python list. This choice was appropriate because:

- a Python list naturally supports dynamic array behavior
- parent and child indices can be computed efficiently
- heap operations such as insertion and extraction are straightforward to implement
- the array-based representation is memory-efficient and avoids the overhead of linked nodes

A max-heap was selected because the scheduling system needed to process the highest-priority task first.

6. Priority Queue Operations and Complexity Analysis

6.1 Insert Operation

The insert(task) operation adds a new task to the end of the heap and then restores the heap property using heapify-up. In the worst case, the inserted element may move from the bottom of the heap to the root.

Therefore, the time complexity is:

$$O(\log n)$$

6.2 Extract Maximum Operation

The extract_max() operation removes the root of the max-heap, which contains the highest-priority task. The last element is moved to the root position, and heapify-down is used to restore the heap property.

The time complexity is:

$$O(\log n)$$

6.3 Peek Operation

The peek() operation returns the highest-priority task without removing it. Since this task is stored at the root of the heap, the operation requires only direct access.

The time complexity is:

$$O(1)$$

6.4 Is_Empty Operation

The is_empty() operation checks whether the heap contains any elements.

The time complexity is:

$$O(1)$$

6.5 Display Operation

The display() operation prints all tasks currently stored in the queue.

The time complexity is:

$$O(n)$$

6.6 Priority Update (Conceptual)

Operations such as increasing or decreasing a task's priority can be performed in:

$$O(\log n)$$

These operations involve repositioning the task within the heap.

7. Scheduling Results and Interpretation

The output of the program showed that tasks were processed in descending order of priority. The inserted tasks were:

- Finish assignment — priority 5
- Reply to emails — priority 2
- Prepare presentation — priority 4
- Attend meeting — priority 3
- Emergency bug fix — priority 10

The scheduler correctly identified Emergency bug fix as the highest-priority task and processed tasks in this order:

1. Emergency bug fix — 10
2. Finish assignment — 5

3. Prepare presentation — 4
4. Attend meeting — 3
5. Reply to emails — 2

This confirms that the heap-based priority queue maintained the max-heap property correctly after each insertion and extraction.

8. Conclusion

In conclusion, heap data structures play a critical role in efficient algorithm design. Heapsort offers consistent $O(n\log n)$performance and operates in-place, making it suitable for memory-constrained environments. However, empirical results showed that Quicksort and Merge Sort often perform better in practice due to lower overhead and better memory behavior.

The priority queue implementation demonstrated how heaps can be used effectively in scheduling systems, ensuring that high-priority tasks are processed first. This highlights the practical importance of heaps in real-world applications such as operating systems, networking, and resource management.

Overall, this assignment provided a comprehensive understanding of heap structures, their performance characteristics, and their applications in solving real-world problems.

9. References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Python*. Wiley.

Weiss, M. A. (2013). *Data structures and algorithm analysis in C++* (4th ed.). Pearson.